Grafakos Classical Fourier Analysis Benchmark

<table>
<tr><th>Natural language statement</th><th>Domain</th><th>GroundTruth</th></tr>
<tr><td>Theorem 1.3.2. Let (X, mu) be a sigma-finite measure space, let (Y, nu) be another measure space, and let 0 < p0 < p1 <= inf. Let T be a sublinear operator defined on L^{p0}(X) + L^{p1}(X) = {f0 + f1 : fj in L^{pj}(X), j = 0, 1} and taking values in the space of measurable functions on Y. Assume that there exist A0, A1 < inf such that ||T(f)||_{L^{p0,inf}(Y)} <= A0||f||_{L^{p0}(X)} for all f in L^{p0}(X), and ||T(f)||_{L^{p1,inf}(Y)} <= A1||f||_{L^{p1}(X)} for all f in L^{p1}(X). Then for all p0 < p < p1 and for all f in Lp(X) we have the estimate ||T(f)||_{Lp(Y)} <= A||f||_{Lp(X)}, where A = 2[p/(p - p0) + p/(p1 - p)]^{1/p} A0^{(p0/p)(p1-p)/(p1-p0)} A1^{(p1/p)(p-p0)/(p1-p0)}.</td><td>Fourier analysis</td><td><pre>
def IsSublinearOperator {X : Type u} {Y : Type v}
    (T : (X -> C) -> Y -> C) : Prop :=
  T 0 = 0 /\
    (forall f g x, ||T (f + g) x|| <= ||T f x|| + ||T g x||) /\
    forall c : C, forall f x, ||T (c * f) x|| = ||c|| * ||T f x||

def HasWeakType {X : Type u} {Y : Type v} [MeasurableSpace X]
[MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) (T : (X -> C) -> Y -> C)
    (p : R) (C : R>=0inf) : Prop :=
  forall f : X -> C, MemLp f (ENNReal.ofReal p) mu -> forall alpha : R, 0 <
  alpha ->
    nu {y | ENNReal.ofReal alpha < ||T f y||e} <=
      ENNReal.rpow (C * eLpNorm f (ENNReal.ofReal p) mu / ENNReal.ofReal
      alpha) p

def HasStrongType {X : Type u} {Y : Type v} [MeasurableSpace X]
[MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) (T : (X -> C) -> Y -> C)
    (p q : R>=0inf) (C : R>=0inf) : Prop :=
  forall f : X -> C, MemLp f p mu ->
    MemLp (T f) q nu /\ eLpNorm (T f) q nu <= C * eLpNorm f p mu

theorem grafakos_1_3_2_marcinkiewicz_interpolation
    {X : Type u} {Y : Type v} [MeasurableSpace X] [MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) [SigmaFinite mu]
    (T : (X -> C) -> Y -> C) {p0 p1 p : R} {A0 A1 : R>=0inf}
    (hp : 0 < p0 /\ p0 < p /\ p < p1)
    (hA0 : A0 < inf) (hA1 : A1 < inf)
    (hT : IsSublinearOperator T) (h0 : HasWeakType mu nu T p0 A0)
    (h1 : HasWeakType mu nu T p1 A1)
    (hmeas : forall f, AEStronglyMeasurable f mu -> AEStronglyMeasurable (T
    f) nu) :
    HasStrongType mu nu T (ENNReal.ofReal p) (ENNReal.ofReal p)
      (2 * ENNReal.rpow
        (ENNReal.ofReal (p / (p - p0) + p / (p1 - p))) (1 / p) *
          ENNReal.rpow A0 ((p0 / p) * ((p1 - p) / (p1 - p0))) *
            ENNReal.rpow A1 ((p1 / p) * ((p - p0) / (p1 - p0)))) := by
  sorry
</pre></td></tr>
<tr><td>Theorem 1.3.4. Let (X, mu) and (Y, nu) be two sigma-finite measure spaces. Let T be a linear operator defined on the set of all finitely simple functions on X and taking values in the set of measurable functions on Y. Let 1 <= p0, p1, q0, q1 <= inf and assume that ||T(f)||_{L^{q0}} <= M0||f||_{L^{p0}} and ||T(f)||_{L^{q1}} <= M1||f||_{L^{p1}} for all finitely simple functions f on X. Then for all 0 < theta < 1 we have ||T(f)||_{L^q} <= M0^{1-theta}M1^theta||f||_{L^p} for all finitely simple functions f on X, where 1/p = (1-theta)/p0 + theta/p1 and 1/q = (1-theta)/q0 + theta/q1. Consequently, when p < inf, by density, T has a unique bounded extension from Lp(X, mu) to L^q(Y, nu) when p and q are as above.</td><td>Fourier analysis</td><td><pre>
def HasStrongType {X : Type u} {Y : Type v} [MeasurableSpace X]
[MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) (T : (X -> C) -> Y -> C)
    (p q : R>=0inf) (C : R>=0inf) : Prop :=
  forall f : X -> C, MemLp f p mu ->
    MemLp (T f) q nu /\ eLpNorm (T f) q nu <= C * eLpNorm f p mu

theorem grafakos_1_3_4_riesz_thorin_interpolation
    {X : Type u} {Y : Type v} [MeasurableSpace X] [MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) (T : (X -> C) ->l[C] (Y -> C))
    {p0 p1 q0 q1 p q : R>=0inf} {theta : R} {M0 M1 : R>=0inf}
    (htheta : 0 < theta /\ theta < 1)
    (hexponents : 1 <= p0 /\ 1 <= p1 /\ 1 <= q0 /\ 1 <= q1)
    (hM0 : M0 < inf) (hM1 : M1 < inf)
    (hp : p^-1 = ENNReal.ofReal (1 - theta) * p0^-1 + ENNReal.ofReal theta
    * p1^-1)
    (hq : q^-1 = ENNReal.ofReal (1 - theta) * q0^-1 + ENNReal.ofReal theta
    * q1^-1)
    (h0 : HasStrongType mu nu T p0 q0 M0)
    (h1 : HasStrongType mu nu T p1 q1 M1) :
    HasStrongType mu nu T p q
      (ENNReal.rpow M0 (1 - theta) * ENNReal.rpow M1 theta) := by
  sorry
</pre></td></tr>
<tr><td>Theorem 2.1.6. The uncentered and centered Hardy-Littlewood maximal operators M and M^c map L1(Rn) to L^{1,inf}(Rn) with constant at most 3n and also Lp(Rn) to Lp(Rn) for 1 < p < inf with constant at most 3^{n/p}p(p-1)^{-1}. For any f in L1(Rn) we also have |{M(f)>alpha}| <= (3n/alpha)integral_{M(f)>alpha}|f(y)|dy.</td><td>Fourier analysis</td><td><pre>
noncomputable def hardyLittlewoodMaximal (n : N)
    (f : EuclideanSpace R (Fin n) -> C) (x : EuclideanSpace R (Fin n)) :
    R>=0inf :=
   y : EuclideanSpace R (Fin n), r : {r : R // 0 < r},
     (_ : x in ball y r),
      (integral^- z in ball y r, ||f z||e) / volume (ball y r)
</pre></td></tr>
</table>

<table>
<tr><th>Natural language statement</th><th>Domain</th><th>GroundTruth</th></tr>
<tr><td></td><td></td><td>noncomputable def hardyLittlewoodCenteredMaximal (n : N)
    (f : EuclideanSpace R (Fin n) -> C) (x : EuclideanSpace R (Fin n)) :
    R>=0inf :=
   r : {r : R // 0 < r},
    (integral^- z in ball x r, ||f z||e) / volume (ball x r)

theorem grafakos_2_1_6_hardy_littlewood_maximal {n : N} :
    let operators := ({hardyLittlewoodMaximal n,
      hardyLittlewoodCenteredMaximal n} : Set ((EuclideanSpace R (Fin n) ->
      C) ->
        EuclideanSpace R (Fin n) -> R>=0inf))
    (forall M in operators, forall f : EuclideanSpace R (Fin n) -> C, MemLp
    f 1 volume ->
      forall alpha : R, 0 < alpha -> volume {x | ENNReal.ofReal alpha < M f
      x} <=
        ENNReal.ofReal (3 ^ n / alpha) *
          integral^- x in {x | ENNReal.ofReal alpha < M f x}, ||f x||e) /\
    forall M in operators, forall p : R, 1 < p -> forall f : EuclideanSpace
    R (Fin n) -> C,
      MemLp f (ENNReal.ofReal p) volume ->
        ENNReal.rpow (integral^- x, ENNReal.rpow (M f x) p) (1 / p) <=
          ENNReal.ofReal (3 ^ ((n : R) / p) * p / (p - 1)) *
            eLpNorm f (ENNReal.ofReal p) volume := by
  sorry</td></tr>
<tr><td>Theorem 2.2.14. Given f, g, and h in S(Rn), we have (1) integralRn f(x)g(x) dx = integralRn f(x)g(x) dx; (2) (Fourier Inversion) (f)\/ = f = (f\/); (3) (Parseval's relation) integralRn f(x)h(x) dx = integralRn f(xi)h(xi) dxi; (4) (Plancherel's identity) ||f||_{L2} = ||f||_{L2} = ||f\/||_{L2}; (5) integralRn f(x)h(x) dx = integralRn f(x)h\/(x) dx.</td><td>Fourier analysis</td><td>theorem grafakos_2_2_14_fourier_identities_on_schwartz
    {n : N} (f g h : S(EuclideanSpace R (Fin n), C)) :
    ((integral x, f x * F g x) = integral x, F f x * g x) /\
      F^- (F f) = f /\ F (F^- f) = f /\
      (integral x, star (F f x) * F g x) = integral x, star (f x) * g x /\
      eLpNorm (fun x => F f x) 2 volume = eLpNorm f 2 volume /\
      eLpNorm (fun x => F^- f x) 2 volume = eLpNorm f 2 volume /\
      (integral x, f x * h x) = integral x, F f x * F^- h x := by
  sorry</td></tr>
<tr><td>Proposition 2.2.16 (Hausdorff-Young inequality). For every function f in Lp(Rn) we have the estimate ||f||_{p'} <= ||f||_p whenever 1 <= p <= 2.</td><td>Fourier analysis</td><td>theorem grafakos_2_2_16_hausdorff_young
    {n : N} {p : R} {f : EuclideanSpace R (Fin n) -> C}
    (hp : 1 <= p /\ p <= 2) (hf : MemLp f (ENNReal.ofReal p) volume) :
    let conjugateExponent : R>=0inf := if p = 1 then inf else
    ENNReal.ofReal (p / (p - 1))
    exists F : EuclideanSpace R (Fin n) -> C,
      IsLpFourierTransform (ENNReal.ofReal p) conjugateExponent f F /\
      MemLp F conjugateExponent volume /\
        eLpNorm F conjugateExponent volume <= eLpNorm f (ENNReal.ofReal p)
        volume := by
  sorry</td></tr>
<tr><td>Theorem 3.2.8 (Poisson summation formula). Let f be a continuous function on Rn which satisfies |f(x)| <= C(1+|x|)^{-n-delta} for some C,delta > 0 and whose Fourier transform f restricted on Zn satisfies sum_{minZn}|f(m)| < inf. Then for all x in Rn, sum_{minZn} f(m)e^{2piim.x} = sum_{kinZn} f(x+k), and in particular sum_{minZn} f(m) = sum_{kinZn} f(k).</td><td>Fourier analysis</td><td>theorem grafakos_3_2_8_poisson_summation
    {n : N} {f : EuclideanSpace R (Fin n) -> C}
    (hf : Continuous f /\ Integrable f /\
      (exists C delta : R, 0 < C /\ 0 < delta /\ forall x,
        ||f x|| <= C * (1 + ||x||) ^ (-(n : R) - delta)) /\
      Summable fun m : Fin n -> Z => F f ((WithLp.toLp 2 fun i => (m i :
      R)))) :
    (forall x : EuclideanSpace R (Fin n),
      sum' m : Fin n -> Z, F f ((WithLp.toLp 2 fun i => (m i : R))) *
          Complex.exp (2 * Real.pi * Complex.I * (sum i, (m i : C) * (x i :
        C))) =
        sum' k : Fin n -> Z, f (x + (WithLp.toLp 2 fun i => (k i : R)))) /\
      (sum' m : Fin n -> Z, F f ((WithLp.toLp 2 fun i => (m i : R)))) =
        sum' k : Fin n -> Z, f ((WithLp.toLp 2 fun i => (k i : R))) := by
  sorry</td></tr>
<tr><td>Theorem 4.1.1. For R > 0 and m in Zn, let a(m, R) be complex numbers such that (i) for every R > 0 there is a qR such that a(m, R) = 0 if |m| > qR; (ii) there is an M0 < inf such that |a(m, R)| <= M0 for all m in Zn and all R > 0; (iii) for each m in Zn, the limit of a(m, R) exists as R -> inf and limR->inf a(m, R) = am. Let 1 <= p < inf. For f in Lp(Tn) and x in Tn define S_R(f)(x) = sum_{minZn} a(m, R)f(m)e^{2piim.x}, noting</td><td>Fourier analysis</td><td>def HasStrongType {X : Type u} {Y : Type v} [MeasurableSpace X]
[MeasurableSpace Y]
    (mu : Measure X) (nu : Measure Y) (T : (X -> C) -> Y -> C)
    (p q : R>=0inf) (C : R>=0inf) : Prop :=
  forall f : X -> C, MemLp f p mu ->
    MemLp (T f) q nu /\ eLpNorm (T f) q nu <= C * eLpNorm f p mu</td></tr>
</table>

Natural language statement	Domain	GroundTruth

that the sum is well defined because of (i). Also, for h in Cinf(Tn) define A(h)(x) = sum_{minZn} amh(m)e^{2piim.x}. Then for all f in Lp(Tn) the sequence S_R(f) converges in Lp as R -> inf if and only if there exists a constant K < inf such that supR>0 ||S_R||_{Lp->Lp} <= K. Furthermore, if this holds, then for the same constant K we have sup_{hinCinf, h!=0} ||A(h)||_{Lp}/||h||_{Lp} <= K, and then A extends to a bounded operator A from Lp(Tn) to itself; moreover, for every f in Lp(Tn) we have that S_R(f) -> A(f) in Lp as R -> inf.		`noncomputable def torusCharacter {n : N} (m : Fin n -> Z)`
		`    (x : Fin n -> AddCircle (1 : R)) : C :=`
		`  prod i, fourier (m i) (x i)`
		
		`noncomputable def torusFourierCoefficient {n : N}`
		`    [MeasurableSpace (Fin n -> AddCircle (1 : R))]`
		`    (mu : Measure (Fin n -> AddCircle (1 : R)))`
		`    (f : (Fin n -> AddCircle (1 : R)) -> C) (m : Fin n -> Z) : C :=`
		`  integral x, star (torusCharacter m x) * f x dmu`
		
		`theorem grafakos_4_1_1_torus_summability_uniform_boundedness`
		`    {n : N}`
		`    (a : R -> (Fin n -> Z) -> C) (aLimit : (Fin n -> Z) -> C)`
		`    (hfinite : forall R, 0 < R -> (Function.support (a R)).Finite)`
		`    (hbounded : exists M : R, 0 <= M /\ forall R m, 0 < R -> ||a R m|| <= M)`
		`    (htendsto : forall m, Tendsto (fun R => a R m) atTop (nhds (aLimit m)))`
		`    {p : R} (hp : 1 <= p) :`
		`    let mu : Measure (Fin n -> AddCircle (1 : R)) := volume`
		`    let S := fun R (f : (Fin n -> AddCircle (1 : R)) -> C)`
		`      (x : Fin n -> AddCircle (1 : R)) =>`
		`        sum' m, a R m * torusFourierCoefficient mu f m * torusCharacter m x`
		`    let formalLimit := fun (f : (Fin n -> AddCircle (1 : R)) -> C)`
		`      (x : Fin n -> AddCircle (1 : R)) =>`
		`        sum' m, aLimit m * torusFourierCoefficient mu f m * torusCharacter`
		`        m x`
		`    ((forall f, MemLp f (ENNReal.ofReal p) mu ->`
		`        exists g, MemLp g (ENNReal.ofReal p) mu /\`
		`          Tendsto (fun R => eLpNorm (S R f - g) (ENNReal.ofReal p) mu)`
		`        atTop (nhds 0)) <->`
		`      exists C : R>=0inf, C < inf /\ forall R, 0 < R ->`
		`        HasStrongType mu mu (S R) (ENNReal.ofReal p) (ENNReal.ofReal p) C)`
		`        /\`
		`    forall C : R>=0inf,`
		`      (forall R, 0 < R -> HasStrongType mu mu (S R) (ENNReal.ofReal p)`
		`      (ENNReal.ofReal p) C) ->`
		`        exists A : ((Fin n -> AddCircle (1 : R)) -> C) ->`
		`            (Fin n -> AddCircle (1 : R)) -> C,`
		`          HasStrongType mu mu A (ENNReal.ofReal p) (ENNReal.ofReal p) C /\`
		`          (forall h, MemLp h (ENNReal.ofReal p) mu ->`
		`            Summable (fun m => fun x =>`
		`              aLimit m * torusFourierCoefficient mu h m * torusCharacter m`
		`        x) ->`
		`              A h = formalLimit h) /\`
		`          forall f, MemLp f (ENNReal.ofReal p) mu ->`
		`            Tendsto (fun R => eLpNorm (S R f - A f) (ENNReal.ofReal p) mu)`
		`        atTop (nhds 0) := by`
		`  sorry`

Theorem 4.3.15. For every 1 < p < inf there exists a finite constant Cp such that for all f in C0inf(R) we have ||C**(f)||_{Lp(R)} <= Cp||f||_{Lp(R)}, where C**(f)(x) = supR>0 |integral_{|xi|<=R} f(xi)e^{2piixxi} dxi| is the Carleson operator.	Fourier analysis	`noncomputable def carlesonHuntMaximal (f : S(R, C)) (x : R) : R>=0inf :=`
		`   R : {R : R // 0 < R},`
		`    ||integral xi in Set.Icc (-R.1) R.1,`
		`      F f xi * Complex.exp (2 * Real.pi * Complex.I * xi * x)||e`
		
		`theorem grafakos_4_3_15_carleson_hunt_line {p : R} (hp : 1 < p) :`
		`    exists C : R>=0inf, C < inf /\ forall f : S(R, C),`
		`      ENNReal.rpow (integral^- x, ENNReal.rpow (carlesonHuntMaximal f x) p)`
		`      (1 / p) <=`
		`        C * eLpNorm (f : R -> C) (ENNReal.ofReal p) volume := by`
		`  sorry`

Theorem 5.3.1. Let f in L1(Rn) and alpha > 0. Then there exist functions g and b on Rn such that f = g+b; ||g||_{L1} <= ||f||_{L1} and ||g||_{Linf} <= 2nalpha; b = sum_j b_j where each b_j is supported in a dyadic cube Q_j, the cubes are disjoint, integral_{Q_j}b_j(x)dx=0, ||b_j||_{L1}<=2^{n+1}alpha|Q_j|, and sum_j |Q_j|<=alpha^{-1}||f||_{L1}.	Fourier analysis	`structure DyadicCube (n : N) where`
		`  scale : Z`
		`  corner : Fin n -> Z`
		
		`def DyadicCube.carrier {n : N} (Q : DyadicCube n) : Set (EuclideanSpace R`
		`(Fin n)) :=`
		`  {x | forall i,`
		`    (Q.corner i : R) * 2 ^ Q.scale <= x i /\`
		`      x i < (Q.corner i + 1 : Z) * 2 ^ Q.scale}`
		
		`theorem grafakos_5_3_1_calderon_zygmund_decomposition`

Natural language statement	Domain	GroundTruth

		`{n : N} {f : EuclideanSpace R (Fin n) -> C} {alpha : R}`
		`(hf : MemLp f 1 volume) (halpha : 0 < alpha) :`
		`exists g b : EuclideanSpace R (Fin n) -> C,`
		`exists (J : Set N) (Q : J -> DyadicCube n)`
		`(bad : J -> EuclideanSpace R (Fin n) -> C),`
		`f = g + b /\ HasSum bad b /\ eLpNorm g 1 volume <= eLpNorm f 1 volume /\`
		`eLpNorm g inf volume <= ENNReal.ofReal (2 ^ n * alpha) /\`
		`(Pairwise fun i j => Disjoint (Q i).carrier (Q j).carrier) /\`
		`(forall j, Function.support (bad j) subset (Q j).carrier) /\`
		`(forall j, integral x, bad j x = 0) /\`
		`(forall j, eLpNorm (bad j) 1 volume <= ENNReal.ofReal (2 ^ (n + 1) * alpha) * volume (Q j).carrier) /\`
		`sum' j : J, volume (Q j).carrier <= eLpNorm f 1 volume / ENNReal.ofReal alpha := by`
		`sorry`

Theorem 5.6.6. For 1 < p,r < inf the Hardy-Littlewood maximal function M satisfies the vector-valued inequalities ||(sum_j |M(f_j)|^r)^{1/r}||_{1,inf} <= C_n(1+(r-1)^{-1})||(sum_j |f_j|^r)^{1/r}||_1 and ||(sum_j |M(f_j)|^r)^{1/r}||_p <= C_n c(p,r)||(sum_j |f_j|^r)^{1/r}||_p.	Fourier analysis	`noncomputable def hardyLittlewoodMaximal (n : N)`
		`(f : EuclideanSpace R (Fin n) -> C) (x : EuclideanSpace R (Fin n)) :`
		`R>=0inf :=`
		`y : EuclideanSpace R (Fin n),  r : {r : R // 0 < r},`
		`(_ : x in ball y r),`
		`(integral^- z in ball y r, ||f z||e) / volume (ball y r)`
		
		`theorem grafakos_5_6_6_vector_valued_maximal`
		`{n : N} {p r : R} (hp : 1 < p) (hr : 1 < r) :`
		`let ellNorm := fun (f : N -> EuclideanSpace R (Fin n) -> C) x =>`
		`ENNReal.rpow (sum' j, ENNReal.rpow ||f j x||e r) (1 / r)`
		`let maximalNorm := fun (f : N -> EuclideanSpace R (Fin n) -> C) x =>`
		`ENNReal.rpow (sum' j, ENNReal.rpow (hardyLittlewoodMaximal n (f j) x) r) (1 / r)`
		`exists Cn Cp : R>=0inf, Cn < inf /\ Cp < inf /\`
		`(forall f : N -> EuclideanSpace R (Fin n) -> C, forall alpha : R, 0 < alpha ->`
		`volume {x | ENNReal.ofReal alpha < maximalNorm f x} <=`
		`Cn * ENNReal.ofReal (1 + 1 / (r - 1)) / ENNReal.ofReal alpha *`
		`ENNReal.rpow (integral^- x, ellNorm f x) 1) /\`
		`forall f : N -> EuclideanSpace R (Fin n) -> C,`
		`ENNReal.rpow (integral^- x, ENNReal.rpow (maximalNorm f x) p) (1 / p) <=`
		`Cp * ENNReal.rpow (integral^- x, ENNReal.rpow (ellNorm f x) p) (1 / p) := by`
		`sorry`